



PIMPRI CHINCHWADE EDUCATION TRUST'S.  
**PIMPRI CHINCHWAD COLLEGE OF ENGINEERING**  
(An Autonomous Institute)  
Affiliated to Savitribai Phule Pune University (SPPU)  
ISO 21001:2018 Certified by TUV SUD

**T.Y. B. TECH**

**Year:** 2025 – 26

**Subject:** FSDL

**Name of the Student:** Manasvi Patil

**PRN:** 124B2B027

**Branch:** Computer Engineering

**Division:** C

## Assignment No 6

**Title : Create Online appointment booking application / E-commerce portal for used items sales like car, bike etc. or similar type application using Node.js and Express use any NoSQL data set like MongoDB if required.**

### Objectives:

- To design and develop a web-based application using Node.js and Express.
- To implement a system for booking appointments OR buying/selling used items (cars, bikes, etc.).
- To use MongoDB (NoSQL database) for storing user data, product details, and transactions.
- To understand backend development, API creation, and database integration.
- To provide a user-friendly interface for managing bookings or product listings.

### Outcomes:

- Understand how to build backend applications using Node.js and Express.
- Learn how to connect and use MongoDB database.
- Develop REST APIs for handling user requests.
- Implement CRUD operations (Create, Read, Update, Delete).

### Theory :

The Online Appointment Booking System or E-commerce Portal for Used Items is a web-based application developed using Node.js, Express.js, and MongoDB.

The system allows users to:

- Book appointments (for services like doctor, salon, etc.)  
OR
- Buy and sell used items such as cars, bikes, and other products.

This application follows a client-server architecture, where:

- The client (frontend) sends requests
- The server (Node.js + Express) processes them
- The database (MongoDB) stores and retrieves data

- **Node.js:** A server-side JavaScript runtime used to build fast and scalable backend applications using non-blocking architecture.

- **Express.js:** A lightweight framework of Node.js used to create APIs and handle routing and server requests easily.
- **MongoDB:** A NoSQL database that stores data in flexible JSON-like format, allowing easy scalability and fast access.

**Code :**

### **Index.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>Appointment Booking</title>
  <style>
    body {
      font-family: Arial;
      background: #f2f2f2;
    }

    .container {
      width: 400px;
      margin: 40px auto;
      background: #fff;
      padding: 20px;
      border-radius: 10px;
    }

    h2 {
      text-align: center;
    }

    input, select, textarea {
      width: 100%;
      padding: 10px;
      margin: 8px 0;
      border-radius: 6px;
      border: 1px solid #ccc;
    }

    button {
      width: 100%;
```

```
padding: 12px;
border: none;
border-radius: 6px;
color: white;
font-size: 16px;
margin-top: 10px;
cursor: pointer;
}

.green { background: #4CAF50; }
.blue { background: #2196F3; }
</style>
</head>

<body>

<div class="container">
  <h2>Schedule Appointment</h2>

  <input id="name" placeholder="Name">
  <input id="age" placeholder="Age">
  <input id="phone" placeholder="Phone">
  <input id="email" placeholder="Email">

  <select id="doctor">
    <option>Dr. Sharma</option>
    <option>Dr. Patel</option>
  </select>

  <input type="date" id="date">
  <input type="time" id="time">

  <textarea id="reason" placeholder="Reason"></textarea>

  <button class="green" onclick="book()">Confirm Booking</button>

  <hr>

  <button class="blue" onclick="goToView()">View Appointments</button>
</div>
```

```
<script>
async function book() {
  const data = {
    name: document.getElementById("name").value,
    age: document.getElementById("age").value,
    phone: document.getElementById("phone").value,
    email: document.getElementById("email").value,
    doctor: document.getElementById("doctor").value,
    date: document.getElementById("date").value,
    time: document.getElementById("time").value,
    reason: document.getElementById("reason").value
  };

  const res = await fetch("/book", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  });

  const result = await res.json();

  if (result.success) {
    alert("✅ Booking Successful!");
    window.location = "/view"; // 🔥 redirect
  } else {
    alert(result.message);
  }
}

function goToView() {
  window.location = "/view"; // 🔥 navigation
}
</script>

</body>
</html>
```

## Server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

// 🔗 MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/appointmentDB")
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));

// 📦 Schema
const appointmentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  phone: String,
  email: String,
  doctor: String,
  date: String,
  time: String,
  reason: String
});

const Appointment = mongoose.model("Appointment", appointmentSchema);
```

```
// 🏠 Main Page
```

```
app.get("/", (req, res) => {  
  res.sendFile(__dirname + "/index.html");  
});
```

```
// 📄 Static Appointments Page
```

```
app.get("/view", (req, res) => {
```

```
  const html = `
```

```
    <html>
```

```
    <head>
```

```
      <title>Appointments</title>
```

```
    <style>
```

```
      body { font-family: Arial; background:#f2f2f2; }
```

```
      .box {
```

```
        width:400px;
```

```
        margin:40px auto;
```

```
        background:white;
```

```
        padding:20px;
```

```
        border-radius:10px;
```

```
      }
```

```
      .card {
```

```
        padding:10px;
```

```
        margin:10px 0;
```

```
        border:1px solid #ddd;
```

```
        border-radius:6px;
```

```
      }
```

```

    button {
      padding:10px;
      width:100%;
      background:#2196F3;
      color:white;
      border:none;
      border-radius:5px;
      cursor:pointer;
    }
  </style>
</head>

  <button onclick="window.location='/'">⬅ Back</button>
</div>
</body>
</html>
`;

res.send(html);
});

// ✅ Booking API (Clash Check still active)
app.post("/book", async (req, res) => {
  const { doctor, date, time } = req.body;

  const exists = await Appointment.findOne({ doctor, date, time });

```

```
if (exists) {  
  return res.json({  
    success: false,  
    message: "⚠ Slot already booked! Try another."  
  });  
}
```

```
const newApp = new Appointment(req.body);  
await newApp.save();
```

```
res.json({  
  success: true  
});  
});
```

```
// 🚀 Start Server  
app.listen(3000, () => {  
  console.log("Server running at http://localhost:3000");  
});
```

Output :

Schedule Appointment

Manu

43

8766901254

manasvipatil1405@gmail.com

Dr. Sharma

22 - 04 - 2026

02 : 23

OPD

Confirm Booking

View Appointments

## Appointments

**Manasvi**

Age: 22

Doctor: Dr. Mehta

Date: 22-04-2026

Time: 10:30 AM

**Rahul**

Age: 28

Doctor: Dr. Sharma

Date: 23-04-2026

Time: 11:00 AM

**Priya**

Age: 25

Doctor: Dr. Mehta

Date: 24-04-2026

Time: 02:15 PM

**Amit**

Age: 30

Doctor: Dr. Sharma

Date: 25-04-2026

Time: 09:45 AM

### Conclusion :

The Travel Agency Website developed using Node.js, Express, and MongoDB provides a simple platform for users to explore travel services and submit booking requests online. The system allows efficient storage and management of customer travel information in a NoSQL database. It demonstrates the integration of frontend technologies with backend server and database. Overall, this project helps in understanding the basics of full-stack web development and dynamic web applications.